EE 474 Lab Part 1: Getting Started with Simulink

Contents

1 Introduction **1**
1.1 Dynamic systems and dynamic simulation 1
1.2 Prelab questions 5

2 In Lab **6**
2.1 Getting started 6
2.1.1 Simulink tutorials on CTMS 6
2.2 Simulink Basics Tutorial 7
2.2.1 Additional steps in the Simulink Basics Tutorial 9
2.3 Introduction: Simulink Modeling 11
2.4 Introduction: Simulink Control 13
2.4.1 Determining controller performance measures from the simulated step response 16
2.5 Discovering a controller to meet the requirements 19

3 Write-up **20**
3.1 Principal values and graphs 20
3.2 Questions 20

4 Appendix **20**
4.1 A note on Simulink 20

1 Introduction

1.1 Dynamic systems and dynamic simulation

Dynamic systems are characterized by time-changing variables. Typical examples might be the speed and torque of a motor drive in industrial automation, or the pressure and temperature of a feed material in a chemical process. Precise feedback control can give precision manufacturing for automation tools, accurate and efficient heating and ventilation for building climate control, and quality outputs with minimal waste for chemical processes.

Dynamic simulation refers to mathematically modeling a dynamical system and determining its outputs for a given set of inputs. The first systems for dynamic simulation were mechanical, employed feedback of mechanical torques to accurately simulate the dynamic equations[1], and were used on such problems as aircraft design in the 1930's.

Later, electronic circuits called ***Analog Computers*** were used for dynamic simulation. Digital flight controls for the Boeing 757, for example, were developed in conjunction with a large analog computer simulating the aerodynamics of the aircraft.

Numerical simulation is the process of setting up the modeling equations to model system response using a digital computer, using a tool such as Simulink. Today, essentially all dynamic simulations are done numerically.

When analyzing a dynamic system, such as a motor drive or chemical process, there are three broad approaches: Theory, Experiment and Simulation. They are interconnected and each has a role to play, and you will be working with each in EE 474.

In a theoretical approach, one develops the mathematical model of the system, and uses theoretically proven properties of the model to analyze or engineer system characteristics. Examples are establishing stability from the pole locations or steady-state error using the final-value theorem. Theory has the advantage of providing results that apply to a broad class of systems, but also has the limitations that models may not be precisely known, or may not precisely fit the assumptions of the theory.

In an experimental approach, one builds a test model of the system and evaluates its performance, such as a pre-production aircraft or pilot test of a chemical process. Experimentation has the advantage that all aspects of the system are included, whether or not they are well understood, but also has the limitations of investigating only a single realization of the system, and of potentially being very expensive.

The third approach is simulation. In a simulation approach, one starts with the mathematical model, as with theory, but rather than proving or using theoretical properties of the model, one quantitatively evaluates the modeling equations to determine the response of the system to a specific set of inputs. Dynamic systems involve variables that change over time (such as speed or pressure), and so involve differential or difference equations. Today, the differential equations of the model are evaluated by ***numerical integration*** on digital computer, which is the process of breaking up the time axis into small steps and computing the response of the system during each small step. The total response of the system is approximately given by the accumulation of the small step-wise increments. By using numerical techniques, such as the ***Runge-Kutta*** method[2], and small step sizes, the numerical integration can accurately approximate the true (but unknown) solution of the differential equations.

Simulation has the advantage that it is widely applicable (it can be used whenever we have a sufficiently valid set of modeling equations), and it is generally inexpensive. Simulation can also be used to study a range of conditions or system designs. It is not unusual for 100's or 1,000's of "simulation-runs" to be used to validate a design in a large project.

However, simulation has the limitation of being only as good as the model, and each "simulation-run" indicates the response of the system to only one set of inputs and initial conditions.

---

1. https://en.wikipedia.org/wiki/Differential_analyser

2. https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta_methods, (used by Simulink)

The advantages and disadvantages of Theory, Experiment and Simulation are summarized in table 1.

- Theory
  - Advantages
    - Very general, applies to all systems meeting the hypotheses of each theorem
    - Can make broad statements, such as "all systems with a given structure are stable"
    - Least expensive
  - Disadvantages
    - Only certain results are available (for example, there is no equation giving the peak overshoot for a higher-order linear system, this is generally found by simulation)
    - Only limited results are available for nonlinear systems
    - Can not consider any aspect of the system that is not understood
- Experiment
  - Advantages
    - Detailed results available
    - It considers all aspects of the system, even those that are not well understood
  - Disadvantages
    - Can be costly and time consuming
    - Only considers the system configurations for which an experimental test bed is built
    - Only provides results on those configurations, inputs and initial conditions that are tested
- Simulation
  - Advantages
    - Inexpensive
    - Can be many times faster than the actual system (for example, we can simulate aircraft landings for many combinations of weight and winds much faster than testing actual landings)
    - Can address nonlinear systems, if we can write the modeling equations we can simulate the system
  - Disadvantages
    - Like theory, simulation can not consider any aspect of the system that is not understood
    - Like experiment, only provides results on those configurations, inputs and initial conditions that are tested
    - Many systems are ***continuous time*** (that is, governed by differential equations). Numerical (digital) simulations only approximate the continuous-time equations. In some cases, the numerical approximation to the continuous differential equations introduces errors.

Table 1: Advantages and disadvantages of Theory, Experiment and Simulation.

1.2 Prelab questions

- Answer these questions as the prelab write-up.
- Note: For all questions throughout the labs, if you express a quantity in the answer, ***indicate the units***.

Question 1. What aspect of dynamic systems corresponds to the fact that they are often described by differential equations ?

Question 2. Name three approaches to evaluating dynamic system characteristics.

Question 3. Name one important limitation of each of the three approaches.

Question 4. If the goal is to find the response of a nonlinear system to a specific input, which of the approaches can be used ?

Question 5. If the goal is to make general statements about the system response, such as determining whether the system is stable for all possible initial conditions, which of the approaches can be used ?

Question 6. If the available modeling information is incomplete, which of the approaches can be used ?

2 In Lab

In Lab 1 you will work through an exercise in the Control Tutorials for Matlab and Simulink (CTMS). The CTMS was built (with support from the NSF) using Matlab 2009a. Mathworks makes two releases of Matlab and Simulink per year. In the Fall of 2017, we are using Matlab release 2017a. Many small characteristics are different between 2009a and 2017a, such as what was called "Start" in 2009a is now called "Run", or in R2017a the Scope tool automatically adjusts the scale when it is first opened. Using CTMS, you will need to be flexible with respect to some of the specifics.

- Simulink is meant to be very easy to use, and you can't hurt it. Don't be shy about trying things.
- Simulink shows short descriptions when the cursor hover over a button.

2.1 Getting started

- Working in groups of 2, log in to an EE 474 lab computer
- Open Matlab (Matlab in Windows 10 takes a very long time, about 1 1/2 minutes, to start)
- Open a web browser, and connect to

  `www.ctms.engin.umich.edu/CTMS`
  – `Under "Basics" choose "Simulink"`

  This will bring you to the unit titled: "Simulink Basics Tutorial."

2.1.1 Simulink tutorials on CTMS

- In this lab, you will work through three tutorials from CTMS:

  1. Simulink Basics Tutorial
  2. Introduction: Simulink Modeling
  3. Introduction: Simulink Control

  They are found at the top-right of the page (under "Basics") and at the bottom-left.

2.2 Simulink Basics Tutorial

- In this part, you will read the tutorial material and carry out all steps. Notes are provided here, with indications of where Simulink has evolved since CTMS was written, and also additional points.
- The section titles in the tutorial are included here with an underline, to facilitate synchronizing these notes with the Simulink tutorial. Where there are notes in this handout below an underlined section title, read the notes while you are working on the items associated with that section title.
- Getting started
  - Create a folder on the PC for this exercise, such as create

    ```
    <user>\Documents\MATLAB\Lab01
    ```

  - In the Matlab command window, change directory to "Lab01"

    ```
    >> cd Lab01
    ```

Starting Simulink

- Go across the top of the simulation window and hover over each button. Find the button which opens the "Library Browser."

Model Files

- Save "simple.mdl" to folder "Lab01"

Basic Elements

Simple Example

Modifying Blocks

- Change the denominator polynomial as indicated.

  Note: try growing and shrinking the transfer function block. Note that the block only displays the transfer function when it has enough space.
- For the scope block
  - Float across each of the buttons at the top of the scope, to see their short help messages. Shortly you will generate some data and have an opportunity to test the scope functions.

Running Simulations

- Download and open `simple2.slx`
- Adjust the size of the Transfer Function block, so that you can see the transfer function.
- Note: "Start" and "Play" (mentioned in CTMS) have been renamed "Run". Auto scaling works a bit differently from the description in CTMS.
- When you open the configuration parameters page, you'll see that there are some changes relative to what is shown in CTMS. Mathworks has endeavored to separate the commonly (and safely) changed parameters from more advanced options.
  - Open the configuration Parameters page, as indicated
  - Compare the "Solver" pane with that shown in CTMS
  - Select "Additional Options" on the up-to-date Configuration Parameters, Solver pane, and compare the options with those shown in CTMS

  The options are essentially the same.
- An important point to keep in mind about numerical simulation is that errors can be introduced in the numeric integration step itself. Example situations that can cause errors would be
  - Using too large a step size with a system that has slow and fast modes
  - Modeling a system with discontinuities in the equations, without taking special steps.

  Experience, or a graduate-level course in the mathematics of numerical methods (such as Math 715 "Numerical Analysis") gives an engineer the background to choose the solver, the step type and some of the advanced (or "Additional") options. The main points to remember here are that
  - Simulation is not 100% reliable (not everything that comes out of a computer is correct !)
  - There is a science of Numerical Analysis, that deals with the mathematical fundamentals of simulating continuous differential equations on digital computers
  - Before simulation results are used in business decisions (such as choosing amongst alternative product designs) the validity of the numerical simulations must be established
  - Correctness is more important than optimality (such as executing the simulation as fast as possible)
- For Simulink, Mathworks' default choices (e.g, ode45 solver with variable step size) will give correct results in most circumstances, and all cases we will encounter in EE 474.

- When making changes to the configuration (such as changing the start and stop times), hit the "Apply" or "OK" button for the new parameters to take affect.

Building Systems

- Build the closed loop model from scratch, as described in the steps, rather than downloading `buildex.slx`. You can look at `buildex.slx`, if you need guidance on a particular point.

Gathering Blocks

- Recall that the "Simulink Library Browser" is opened by from the appropriate icon on the Model Window.

Modify Blocks

- Follow the steps indicated. Adjust the transfer function block to be able to view the transfer function.
- Change the name of the Scope, note that the name of the scope window changes (this can be used to select the scope window in the Windows Task Bar (at the bottom of the screen).

Simulation

- Follow the steps indicated.

Taking Variables from MATLAB

- Follow the steps indicated.

2.2.1 Additional steps in the Simulink Basics Tutorial

- The ***percent overshoot*** (PO) is the percentage by which the peak of the output signal exceeds the output's steady-state value. Defining:

  y_0 Initial value of output $y(t)$, at the start of the transient

  y_{ss} Steady-state value of $y(t)$, in response to a step input

  y_{peak} Peak value of the output, in response to a step input

  then the percent overshoot is given by:

$$PO = 100 \times \frac{y_{peak} - y_{ss}}{y_{ss} - y_0} \tag{1}$$

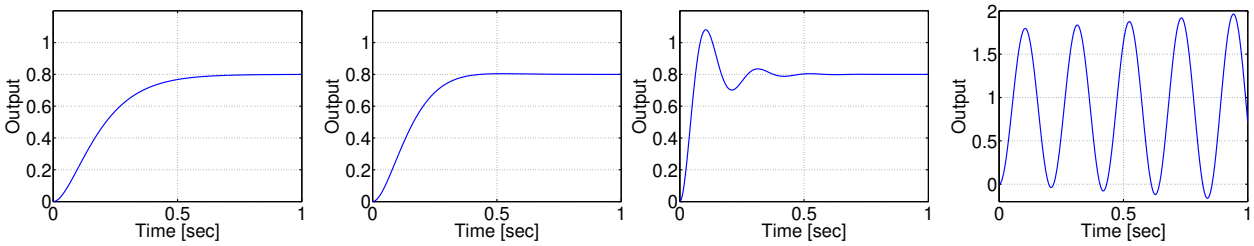


Figure 1: Illustrations of 2nd order response to a step input, for systems with different levels of damping and percent overshoot. *(a)* Overdamped (no overshoot), *(b)* Response with minimal overshoot *(c)* Underdamped (about 20% overshoot), *(d)* Slightly unstable.

1. Modify the PID control block to set the integral gain to 4 (leave the proportional gain as 1, and the derivative gain as 0)

2. By changing K via the Matlab command prompt, explore the response to various values of K (Note, the gain block holds parameter `K` from the step "Taking Variables from MATLAB". When you modify the variable in the Matlab workspace, it is modified in the Simulink model.)

   Note: A system response is ***stable*** when the unforced portion of the response dies away. The responses in figures 1 (a), (b) and (c) are stable. A system response is ***marginally stable*** when it has a steady oscillation (not shown), and a response is ***unstable*** when the oscillation grows over time (figure 1(d)).

   (a) To within +/- 0.1, determine the largest value of K that gives zero overshoot

   (b) To within +/- 0.1, determine the value of K that gives 10% overshoot

   (c) To within +/- 0.1, determine the largest value of K that is stable

2.3 Introduction: Simulink Modeling

- Complete the tutorial activity "Introduction: Simulink Modeling."
- Notes are provided here, read and incorporate these notes, as you are carrying out the sections of the tutorial.

Train system

Free-body diagram and Newton's second law

Constructing the Simulink model

- Build the train model from scratch, as described in the steps, rather than downloading `train.slx`. You can look at `train.slx`, if you need guidance on a particular point.
- For arranging blocks, notice that you can select one or more blocks, and use the arrows to move the blocks by a small increment. Control-Arrow moves the selected blocks by larger steps.

Note: The block diagram must capture the modeling equations, Eqns (1) and (2). A question in the write-up asks you to validate that your correct model implements the modeling equations.

Running the model

- Open the parameters pane of the signal generator and identify each of the parameters. In the future, you will use Simulink blocks without CTMS instructions, so it is good to get in the habit of reading all of the options in the parameter sheets.
- When you get to the step of setting simulation parameters, recall that you can see the name of each button on the model window by hovering over the button with the cursor. Identify the parameters button and select it.
- For your lab write-up, save images of
  - Your model, and
  - The X2 scope output for the step response (Principal Graph)

- Note on printing figures:

  For the scope plot, there are three ways to print:

  1. Select file -> print
  2. Select file -> print to figure
     - Now print the figure through Matlab, such as with

       ```
       >> print('-djpeg', 'x1_dot')
       ```

       to print a jpeg file.
  3. Use the print-screen mechanism (and later trim the desired graphic from the screen shot).

- For the model, on the model window select file -> print and export the model page as a graphic (not as a Simulink model) to a pdf file.

2.4 Introduction: Simulink Control

- For this tutorial, you will follow the steps of the tutorial in sections

  The open-loop plant model

  Implementing a PID controller in Simulink

  Running the closed-loop model

  But then we will do something different for the last two sections. DO NOT DO THESE TWO SECTIONS.

  Extracting a model into MATLAB

  Controller design within Simulink

The open-loop plant model

- The tutorial page gives a link to the completed model. Go over your model very carefully to assure that it is correct.

Implementing a PID controller in Simulink

- In this section, you will encapsulate your train model as a "subsystem," and connect a PID control block to the train. Simulink supports the concept of a subsystem, which is a model with input and output blocks, collapsed into a single block.

  Since subsystems can contain subsystems, it is possible to model complex systems and permit views on the appropriate level of detail.

- Encapsulating the train model as a subsystem:

  - Save your model (file tab in the model window) as "train_cl"

  - Following the instructions in CTMS, change the signal generator and scopes to In1 and Out1 blocks.

    * After you delete the scopes and signal generator and drop In1 and Out1 blocks onto the model sheet, try selecting an In1 or Out1 block, and scooching it on the page with the arrow buttons until the block hooks up to the exiting links.

      If this is difficult, you can also delete the existing links, and make new links.

  - Creating the subsystem:

    The "Create Subsystem" button is accessed by selecting all (cntl-a), then one of :

    1. Select all (cntl-a), then right-click a selected element, or

2. Diagram -> Subsystem & Model Reference -> Create Subsystem from Selection

If needed, adjust the locations of the In1 and Out1 blocks so that the model looks about as shown in the tutorial.

Running the closed-loop model

- Follow the steps to run your closed-loop model, with $k_p = 1.0$.
- If interested, you can a) open the subsystem model, b) change the model by introducing the error, c) re-run the model to see that results match the CTMS figure, d) change the model back.

Do not do CTMS sections "Extracting a model" or "Controller design"

- The techniques used in these sections now requires a special Matlab license. So we will take a different approach.

Controller design for the model train

- Open the parameters sheet for the scope, and set the number of input ports to 3. Three inputs will appear on the scope block, connect them to $r(t)$, $u(t)$ and $y(t)$, as shown in figure 2. Your model should look like that in figure 2.
- In the PID control block, set the k_p and k_i parameters to variables `kp` and `ki`, as seen in figure 3. In the Matlab command window, set

```
>> kp = 1
>> ki = 0
```

- In this exercise, the derivative control will be zero.
- Adjust the signal builder to produce a rising edge at $t = 10$ seconds, and a falling edge at $t = 50$ seconds. The reference signal $r(t)$ is seen in figure 4 as the solid trace.
- To better view the response, adjust the simulation time to 60 seconds (Stop time in the settings pane)

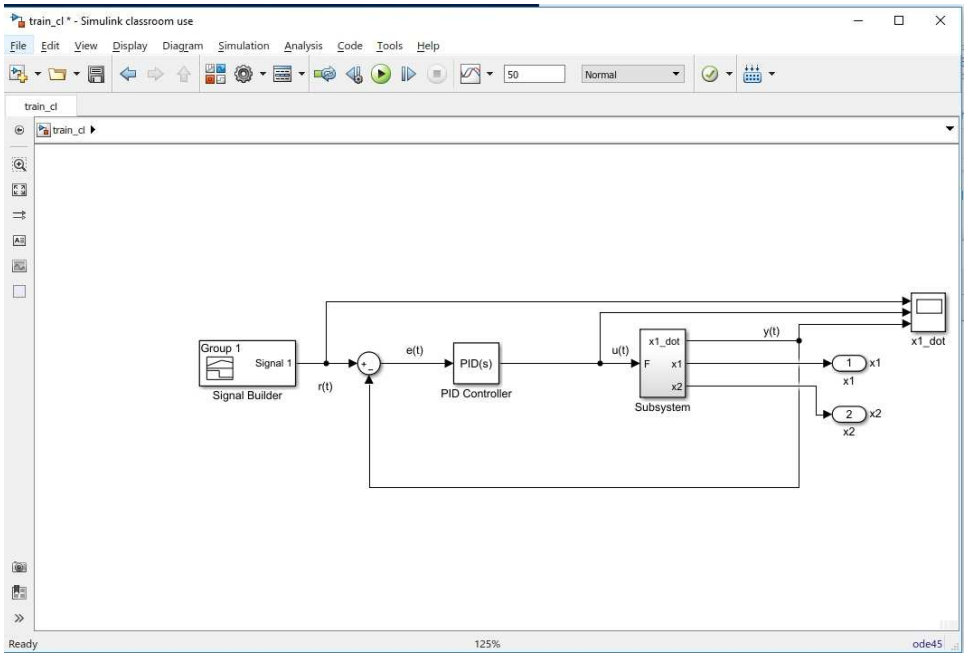


Figure 2: Model train_cl, a simulation diagram with a PID Controller block, a subsystem modeling the train and outputs.

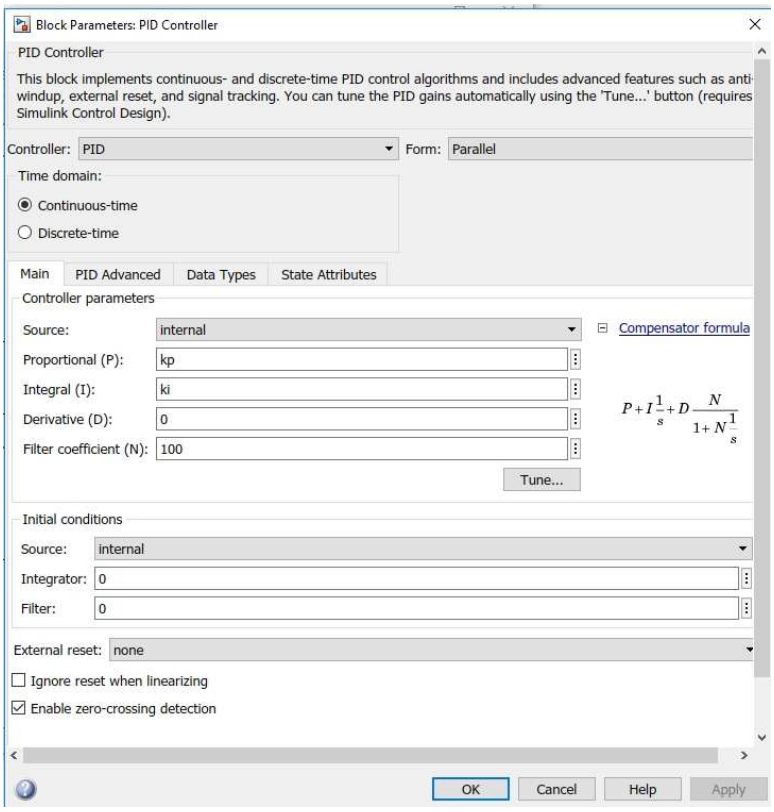


Figure 3: Parameter sheet for the PID Controller block. When variables such as `kp` and `ki` are entered, these values are drawn from the Matlab workspace. The variables must be defined in Matlab, or an error will result.

- Your objective is to tune the controller to achieve the system step-response characteristics of:

  Maximum control effort: $u_{max} \leq 2.0$ [volts]

  Percent overshoot: $PO \leq 10\%$

  Steady-state error: $SSE\,(step) = 0$ [meters/second]

  Rise time: $t_r \leq 2.5$ [seconds]

- These quantities can be observed in the step response plot. With the simulation model setup as in figure 2, the step response will have waveforms that look like those of figure 4. (If it does not, go back and verify the correctness of your train simulation).

  In figure 4, the solid trace is the reference command, $r(t)$, while the dot-dash trace is the output signal, $y(t)$ and the dash-dash trace is the control effort, $u(t)$. (See also figure caption).

  A detail of the step response plot is seen in figure 5.

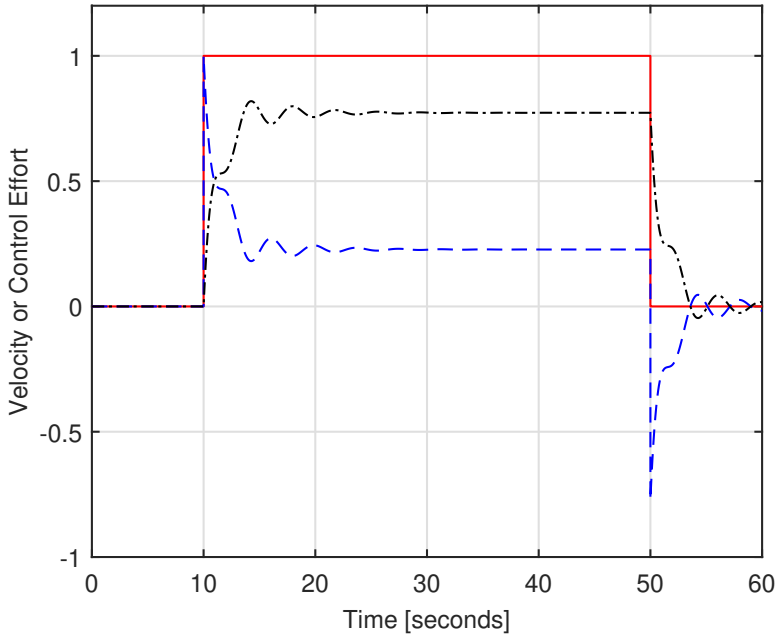


Figure 4: Step response plot. At $t = 20$ [sec], from top to bottom, the traces are solid $(r(t))$, dot-dash $(y(t))$ and dash-dash $(u(t))$.

2.4.1 Determining controller performance measures from the simulated step response

- To get higher resolution in the simulation output data,

- Go into the "Model Configuration Parameters" (gear on Simulink model pane),

  It should show the default options, which are `Variable-step` and `ode45`.

- Open "Additional options"

- Set "Relative tolerance" to `1e-8`

  This will reduce the "step size" and give smoother output curves. Setting this value lower will also lengthen the run time of the simulation, but it runs quite quickly. "Step size" is in quotes, because a variable step size method is being used (`ode45`) so there is no fixed step size.

- Control systems are characterized by up to 20 distinct quantities, some determined from the step response, some from the frequency response, and some from the pole-zero constellation. Here will consider the four that are our design goals.

  - The ***maximum control effort*** is the peak of the absolute value of the dash-dash trace, 1.0 in figure 4.

$$u_{peak} = \max_t \left(|u(t)| \right) \tag{2}$$

  - **Percent overshoot** is defined in Eqn (1) above. Considering figure 4 and the detail in figure 5, one finds that $y_{peak} = 0.82$ [m/s], while $y_{ss} = 0.77$ and $y_0 = 0$ [m/s] (note that the output variable is the train velocity). These values give

$$PO = (0.82 - 0.77) / (0.77 - 0) = 6.5\%$$

    For this lab, taking measurements from the plots is fine (though if you want to log data to the Matlab workspace, and write short routines to do the analysis, that is both quicker to repeat and more accurate). If you take measurements from the plots, for Lab 1, two places of accuracy is sufficient. Use the plot zoom controls to zoom in on the region of the measurement.

  - ***Steady-state error*** to a step input (SSE(step)) is the difference between the steady-state output and the reference input. The error signal in a standard control loop is given by:

$$e(t) = r(t) - y(t) \tag{3}$$

    and so the steady-state error is given by:

$$e_{ss} = r_{ss} - y_{ss} \tag{4}$$

    In figure 4, $e_{ss} = 1.0 - 0.77 = 0.23$ [meter/second].

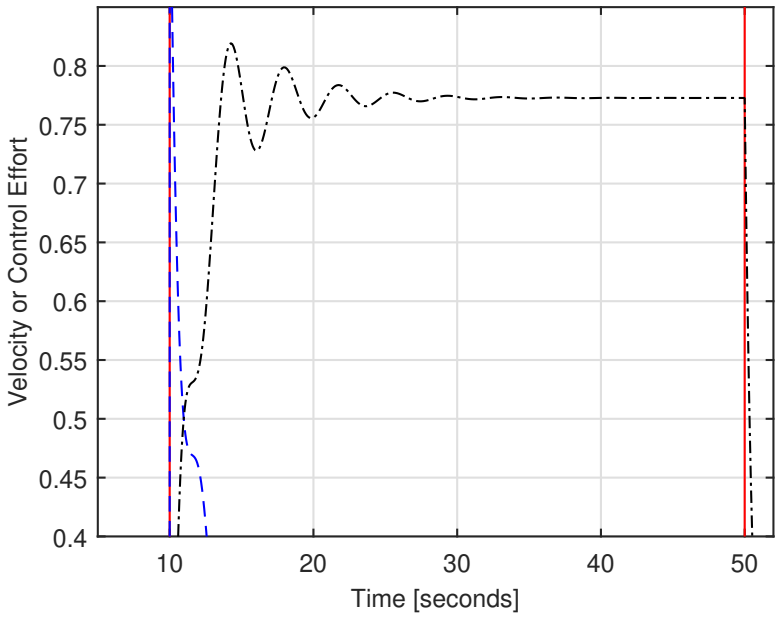


Figure 5: Detail of the step response.

– ***Rise time***, t_r, is the time taken for $y(t)$ to move from 10% of the transient to 90% of the transient (also known as the "10-90 rise time"). Defining t_0, y_0 to be the time and output value of the start of the transient, then the values y_{10} and y_{90} are the defined as the $y(t)$ values at 10% and 90% of the transient, and are given by:

$$y_{10} = y_0 + \frac{1}{10}(y_{ss} - y_0) \tag{5}$$

$$y_{90} = y_0 + \frac{9}{10}(y_{ss} - y_0) \tag{6}$$

The corresponding times are given by:

$$y(t_0 + t_{10}) = y_{10} \tag{7}$$

$$y(t_0 + t_{90}) = y_{90} \tag{8}$$

and the 10-90 rise time is given by

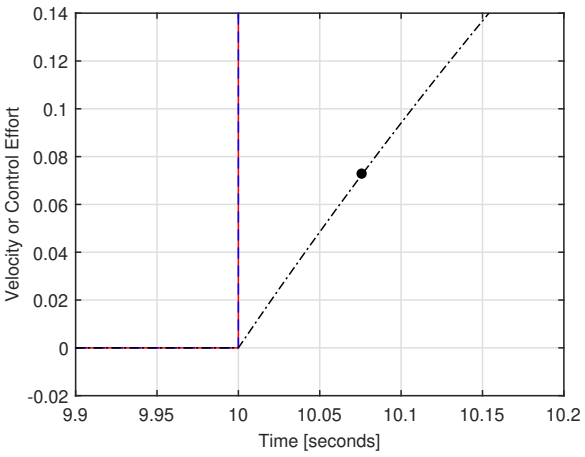
$$t_r = t_{90} - t_{10}\,. \tag{9}$$

From figure 4, $t_0 = 10.0$ [sec], and $y_0 = 0$ [m/s]. $y_{ss} - y_0$ gives 0.77, so $y_{10} = 0.077$ and $y_{90} = 0.70$. Looking at the zoomed-in views in figure 6.

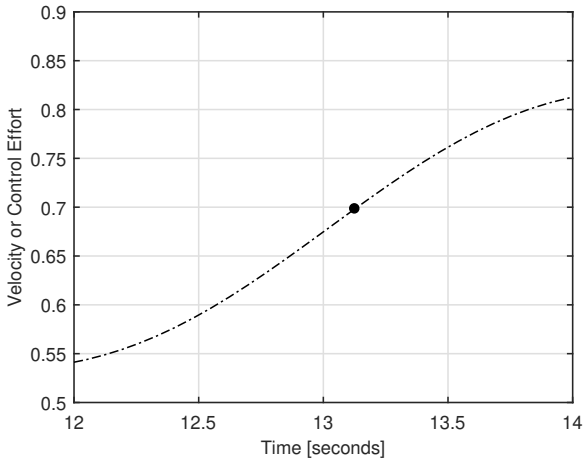
* In figure 6 (a), t_{10} is read as $t_{10} = 10.07$ seconds.
* In figure 6 (b), t_{90} is read as $t_{90} = 13.15$ seconds (these can also be read in a zoomed-in view in Simulink). These values give t_r as

$$t_r = 13.15 - 10.07 = 3.08 \text{ [seconds]} \tag{10}$$

- The results of our first controller trial are summarized in table 2. Our first controller meets the u_{max} and PO requirements, but fails the SSE and t_r requirements.



(a) Crossing $y_{10} = 0.077$



(b) Crossing $y_{90} = 0.70$

Figure 6: Details of the step response, including key points for determining rise time.

Trial	Controller		Response			
#	k_p	k_i	u_{max} [volts]	PO	SSE(step) [m/s]	t_r [sec]
1	1.0	0.0	1.0	6.5%	0.23 [m/s]	3.08 [sec]
⋮	⋮	⋮	⋮	⋮	⋮	⋮

Table 2: Control design results, showing the first iteration. Repeat this table (with more rows) in your notebook. Use the number of trials needed to discover a controller that meets the requirements.

2.5 Discovering a controller to meet the requirements

- Validate that your Simulink model produces the first row of table 2.
- By adjusting k_p and k_i, determine a controller design that meets the four requirements. Suggestion: when exploring the influence of k_p or k_i, introduce large changes, such 1, 3, 10, not 1.0, 1.1, 1.2.
  - Explore the influence of k_p on the four requirements, keeping k_i at zero
  - Setting k_p to a value that does not give overshoot, explore the influence of k_i
  - Find values of k_p and k_i that meet all four requirements.
- During your iterations, record your results in a table like table 2, with sufficient rows.

- For your report, print figures (or grab screen shots)
  - Showing the step response meeting the requirements (a Principal Graph)
  - Showing the model.

3 Write-up

- Write a report, as described in the *"Style Guide for EE 474 Postlab Write-up"*. Include the Principal Values and Graphs and answers to the questions below.

3.1 Principal values and graphs

1. Section 2.3, plot of $X2$ output.
2. Section 2.5,
   (a) Your values for `kp` and `ki` .
   (b) A plot of the data plotted by the scope

3.2 Questions

Question 7. In section 2.2.1, what were the values of K you determined

Question 8. in section 2.5, what are the parameters of your controller.

Question 9. What are the values of maximum control effort, percent overshoot, steady-state error and rise time given by your controller ?

4 Appendix

4.1 A note on Simulink

When you hit "Run" in Simulink a complex chain of actions is set in motion. Simulink writes a program to simulate the system, then compiles and executes that program. When it first came out in 2002, Simulink wrote a Matlab program, which could then be copied, modified and run from the Matlab command window.

For speed, Simulink now writes a C or C++ program, that is compiled with a C compiler included with the tool. In some tests, code running in C is 70 to 200 times faster than code running in Matlab, and complex simulations may require considerable computation time. If you watch closely, you can observe the time required for the build and launch process, especially the first time Simulink runs a simulation being launched.

This design choice permits the C code to be built and used for other targets (types of microprocessor) and used for other purposes (such as actually controlling the ultimate equipment (a train in our case)).

Model-based design refers to developing a real-time application within a Simulink model, validating the controller against a model of the plant, then converting the controller component directly from the model into code that executes on the actual system. An example of control implemented this way is the Space-X Falcon 9 launch vehicle, where flight control is validated in a detailed Simulink model, and then transferred from the Simulink model via code generation to embedded microprocessors that fly the rocket, without being edited by human hands. Model-based design reduces the programming time, since the target controller does not have to be coded separately from the modeling, and, more importantly, reduces the chance of a human error making the flight-version of the system different from the validated-version.

It is interesting to think of Simulink as a graphical programming tool. As you build the model with blocks and links, under the hood you are generating the code that runs the simulation.